

ACTIVE-IA

Devolución de Trabajo Práctico

Materia:	PROG3-M2026 - Programación 3
Comisión:	COMI-14 (2026)
Trabajo:	Evaluación Parcial 2 - JPA: Repositorios y ABM (con validación del modelo TP8)
Alumno:	Damian Eduardo Tristant

Calificación Final: 65/100

EVALUACIÓN POR CRITERIOS

C1: TP8 base: estructura, mapeo y configuración (cimiento del Parcial)

3/4



La estructura base del TP8 está mayormente correcta, incluyendo la superclase Base, las entidades heredando de ella, los enums y la configuración de persistence.xml. Sin embargo, la interfaz Calculable no es implementada explícitamente por Pedido, aunque el método calcularTotal() existe. La principal desviación es la ausencia de la clase JPUtil para centralizar la creación del EntityManagerFactory; en su lugar, el EntityManagerFactory y EntityManager se crean directamente en Main y el EntityManager se pasa a los repositorios, lo cual no sigue la arquitectura propuesta para la gestión del EntityManagerFactory.

C2: Relación Usuario-Pedido (asociación dirigida 1..m)

4/4



La relación Usuario-Pedido está correctamente implementada como una asociación unidireccional 1..m desde Usuario, utilizando Set<Pedido> y la anotación @OneToMany con @JoinColumn en la clase Usuario.

C3: Relación Categoría-Producto (agregación 1..m)

4/4

La relación Categoría-Producto está correctamente implementada como una agregación unidireccional 1..m desde Categoría, utilizando Set<Producto> y la anotación @OneToMany con @JoinColumn en la clase Categoría.

C4: Relación Pedido-DetallePedido (composición 1..m)

4/4

La relación Pedido-DetallePedido está correctamente implementada como una composición 1..m desde Pedido, utilizando Set<DetallePedido> y la anotación @OneToMany con CascadeType.ALL, lo cual es apropiado para la composición.

C5: Relación DetallePedido-Producto (asociación dirigida m..1)

4/4

La relación DetallePedido-Producto está correctamente implementada como una asociación dirigida m..1, con el atributo Producto en DetallePedido anotado con @ManyToOne.

C6: HU-01: BaseRepository<T> (repositorio genérico con CRUD)

7/14

La implementación de BaseRepositoryImpl presenta varias desviaciones significativas de la rúbrica. No se utiliza JPAUtil para obtener el EntityManagerFactory, y el EntityManager se inyecta y se mantiene abierto, en lugar de abrirse y cerrarse en cada método como se especifica. El método guardar utiliza persist en lugar de merge, lo que puede causar problemas al intentar actualizar entidades existentes. buscarPorId no retorna Optional<T> y eliminarLogico no retorna boolean, incumpliendo las firmas requeridas. Además, guardar no incluye un bloque try-catch-finally con rollback para la transacción.

C7: HU-02: CategoriaRepository / ProductoRepository + JPQL tipada

8/10

Los repositorios CategoriaRepositoryImpl y ProductoRepositoryImpl extienden correctamente BaseRepositoryImpl. La consulta buscarPorCategoria en ProductoRepositoryImpl está bien implementada usando JPQL tipada y parámetros nombrados, y filtra correctamente por productos activos de una categoría. Sin embargo, el proyecto no incluye un archivo README.md y el método buscarPorCategoria carece del comentario explicativo requerido para la consulta JPQL.

C8: HU-03: Alta de Categoría (Main)

4/6



La funcionalidad de alta de categoría solicita correctamente el nombre y la descripción, y muestra el ID generado al persistir. La categoría se crea con eliminado = false y createdAt automático. Sin embargo, no se implementa la validación para evitar que el nombre de la categoría esté vacío, lo cual es un requisito.

C9: HU-04: Modificación de Categoría (Main)

4/8



La modificación de categoría muestra los valores actuales y persiste los cambios. Sin embargo, no lista las categorías activas antes de solicitar el ID, lo que dificulta la usabilidad. Además, si se deja un campo en blanco al modificar, el valor anterior se sobrescribe con una cadena vacía en lugar de conservarse, lo cual es un incumplimiento del requisito.

C10: HU-05: Baja lógica de Categoría (Main)

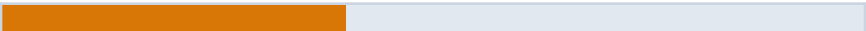
4/6



La baja lógica de categoría funciona correctamente, marcando eliminado = true y excluyendo la categoría de los listados activos. Maneja el caso de ID inexistente. Sin embargo, no se muestra el nombre de la categoría afectada en el mensaje de confirmación, lo cual es un requisito de usabilidad.

C11: HU-06: Alta de Producto (Main)

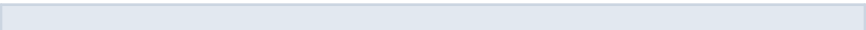
4/10



La funcionalidad de alta de producto lista las categorías activas y solicita los datos del producto. La relación con la categoría se resuelve correctamente a través de la colección en Categoría. Sin embargo, no hay validación para asegurar que el precio sea mayor a 0 y el stock sea mayor o igual a 0. Además, no se muestra el ID generado del producto ni la categoría asignada en el mensaje de confirmación. El manejo de la ausencia de categorías activas podría ser más explícito.

C12: HU-07: Modificación de Producto (Main)

0/8



La modificación de producto muestra los valores actuales. Sin embargo, no lista los productos activos antes de solicitar el ID. Un error crítico es que no se maneja el caso de campos vacíos al intentar parsear Double o Integer, lo que provocará un NumberFormatException si el usuario no ingresa un valor. Además, no se implementan las validaciones para asegurar que el precio sea mayor a 0 y el stock no sea negativo.

C13: HU-08: Baja lógica de Producto (Main)

4/6

La baja lógica de producto funciona correctamente, marcando eliminado = true y excluyendo el producto de los listados activos. Maneja el caso de ID inexistente. Sin embargo, no se muestra el nombre del producto afectado en el mensaje de confirmación.

C14: HU-09: Consulta JPQL - Productos por categoría (Reportes en Main)

11/12

La consulta de productos por categoría funciona correctamente, listando las categorías activas, invocando el método buscarPorCategoría del repositorio y mostrando los resultados formateados, incluyendo el mensaje para categorías sin productos. La consulta JPQL está bien construida. Sin embargo, la opción no está anidada dentro de un submenú de 'Reportes' como se especifica en la rúbrica, sino que se encuentra directamente en el menú principal.

FORTALEZAS

- Correcta implementación de las relaciones JPA 1..m y m..1 con cascada y Set en las entidades.
- Uso de Lombok (@SuperBuilder, @Getter, @Setter, @ToString) para simplificar el código de las entidades.
- La consulta JPQL buscarPorCategoría está bien construida y utiliza TypedQuery con parámetros nombrados.
- La lógica de baja lógica (eliminado = true) está correctamente implementada en BaseRepositoryImpl y utilizada en Main.

RECOMENDACIONES

1. Implementar la clase JPAUtil para centralizar la gestión del EntityManagerFactory y asegurar que cada método del repositorio abra y cierre su propio EntityManager en un bloque try-finally.
2. Ajustar el método guardar en BaseRepositoryImpl para usar em.merge() en lugar de em.persist() para manejar tanto la creación como la actualización de entidades.
3. Mejorar la robustez del Main añadiendo validaciones de entrada para campos numéricos (precio, stock, ID) y cadenas de texto (nombre, descripción) para evitar NumberFormatException y asegurar que los campos no queden vacíos cuando se espera un valor.
4. Asegurar que el método eliminarLogico en BaseRepository retorne boolean y que buscarPorId retorne Optional<T> según las firmas especificadas en la rúbrica.

COMENTARIOS DEL EVALUADOR

El trabajo presenta una base sólida en el mapeo de entidades JPA y la implementación de repositorios genéricos. Sin embargo, existen desviaciones importantes en la gestión del EntityManager y las transacciones, así como en la robustez de la interfaz de usuario en Main con respecto a las validaciones y el manejo de entradas. Se recomienda revisar la implementación de BaseRepositoryImpl y las interacciones en Main para cumplir con todos los requisitos de la rúbrica.